

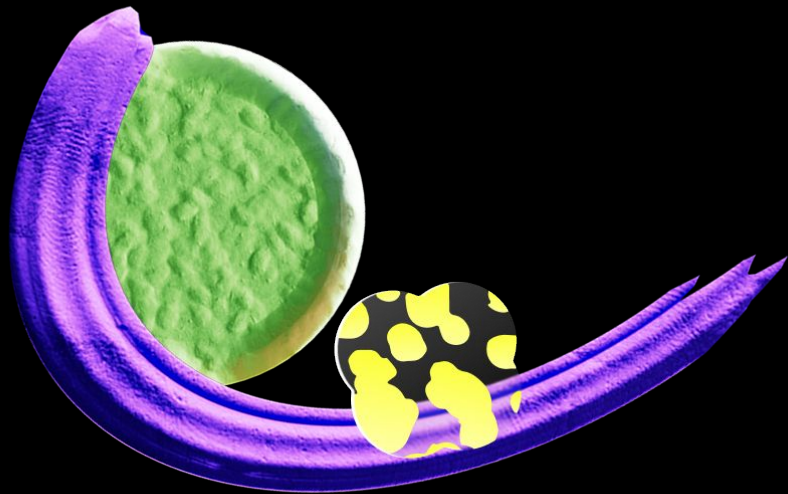
Отметьте студентов в журнале

Архитектура через поведение: BDD, сценарии и спецификации

Михаил Каморин
Tech Lead



Speaker

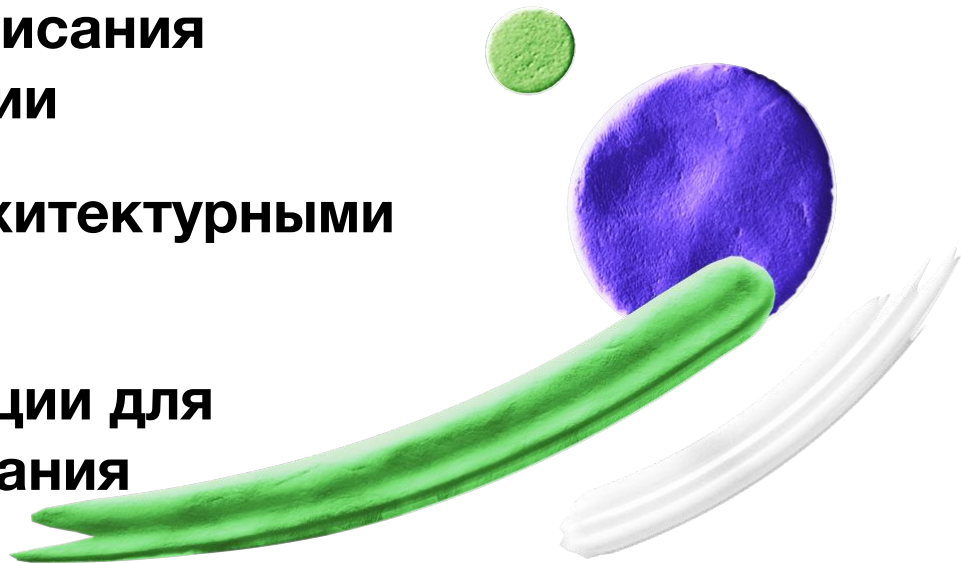


Михаил Каморин
Tech Lead

16 лет в IT, 6 лет вёл
разные курсы в OTUS

Сегодня вы узнаете, как:

- 1 **Использовать BDD для описания требований через сценарии**
- 2 **Связывать сценарии с архитектурными решениями**
- 3 **Использовать спецификации для тестируемого проектирования**



План занятия

- 1 BDD: Поведение как основа проектирования**
- 2 User Stories и Acceptance Criteria**
- 3 Связь поведения и архитектуры**
- 4 Спецификации и тестируемое проектирование**

I. BDD: Поведение как основа проектирования

01 Что такое BDD и как он нам поможет

02 Формат Gherkin для описания сценариев



Behavior-Driven Development

- Корни подхода лежат в Test-Driven Development
- **Ключевая идея:** взять User Story, Acceptance Criteria и формализовать их в бизнес-сценарий
- Сценарий → архитектура → код

Связь между BDD и TDD

Подход	С чего начинаем	Что получаем
TDD	тесты	код
ATDD	Acceptance Criteria	тесты
BDD	BDD-сценарий	тесты

Что мы хотим получить от BDD?

- Как система ведёт себя с точки зрения пользователя?
- Как формализовать поведение так, чтобы его понимали и разработчики, и бизнес?
- Какие архитектурные решения следуют из описанного поведения?

Gherkin – структура описания

- **Given** – имеющийся контекст на момент начала сценария
- **When** – событие (например, действие пользователя), на которое нужна реакция
- **Then** – ожидаемый результат

Feature: Оформление заказа

Scenario: Успешное оформление заказа

Given пользователь добавил товар в корзину

And товар есть в наличии

When пользователь нажимает "Оформить заказ"

Then заказ создаётся со статусом "Новый"

And пользователь видит экран подтверждения

Преимущества BDD

- Ранне выявление зависимостей между компонентами
- Формализация требований к межкомпонентным взаимодействиям
- Прозрачность для бизнеса
- Управление поведением системы

Вопрос для общего обсуждения

Есть ли связь между DDD и BDD? Какая?

2 минуты



Вопрос для общего обсуждения

Каково место агрегатов из DDD в BDD?

2 минуты



II. User Stories и Acceptance Criteria

01 Зачем нужны User Stories?

02 Acceptance Criteria: правила и примеры

03 Связь User Stories и BDD-сценариев



User Story

- **User Story** – заявление о намерениях пользователя
- Отвечает на вопросы:
 - **Кто?** (как правило, роль)
 - **Что** хочет сделать?
 - **Зачем?** (какую бизнес-ценность несёт)

User Story: формат

Как <роль>

я хочу <цель>,

чтобы <бизнес-ценность>

Как пользователь

я хочу положить товар в корзину,

чтобы оформить и оплатить заказ

Acceptance Criteria

- **Acceptance Criteria** – формализация границ поведения
- Отвечает на вопросы:
 - Что считается успехом?
 - Какие варианты поведения допустимы?
 - Какие проверки выполняются?
 - Какие ошибки необходимо обработать?

Example Mapping

Оформ
ление
заказа

Story

Корзина
должна
быть не
пуста

Товар
должен
быть в
наличии

Rule

А если всех
товаров в
корзине нет
в наличии?

Question

А если товар
появляется при
открытой
корзине?

Нет кнопки
“Оформить
заказ” при
пустой
корзине

Если товара
нет, то строка
выделена
красной
рамкой

Example

Связь User Story и BDD-сценария

Как пользователь

я **хочу** оформить возврат товара,

чтобы получить деньги за неподходящий товар

-
- Возврат доступен только для заказов в статусе «Доставлен»
 - Товар не должен иметь признаков использования (нужна проверка)
 - Возврат создаётся со статусом «Ожидает проверки»

Feature: Возврат товара

Scenario: Успешный возврат для доставленного товара

Given заказ находится в статусе "Доставлен"

And товар не имеет признаков использования

When пользователь оформляет возврат

Then возврат создаётся со статусом "Ожидает проверки"

And деньги возвращаются в течение 3 дней

Feature: Возврат товара

Scenario: Попытка возврата использованного товара

Given заказ находится в статусе "Доставлен"

And товар имеет признаки использования

When пользователь оформляет возврат

Then возврат не создаётся

And система сообщает об ошибке пользователю

Практическое значение

- Важность User Story и Acceptance Criteria для архитектуры:
 - Определены зоны ответственности компонентов
 - Помогают выявить интеграции
 - Формализуют поведение для тестирования
 - Снижают риск недопонимания

Вопрос для общего обсуждения

Может ли быть у User Story несколько Acceptance Criteria? А один Acceptance Criteria для нескольких User Story?

2 минуты



Вопрос для общего обсуждения

Как связаны User Story и BDD?

2 минуты



III. Связь поведения и архитектуры

- 01 Архитектурные проекции сценариев
- 02 Диаграммы для оформления проекций
- 03 Связь BDD и Event-Driven Architecture



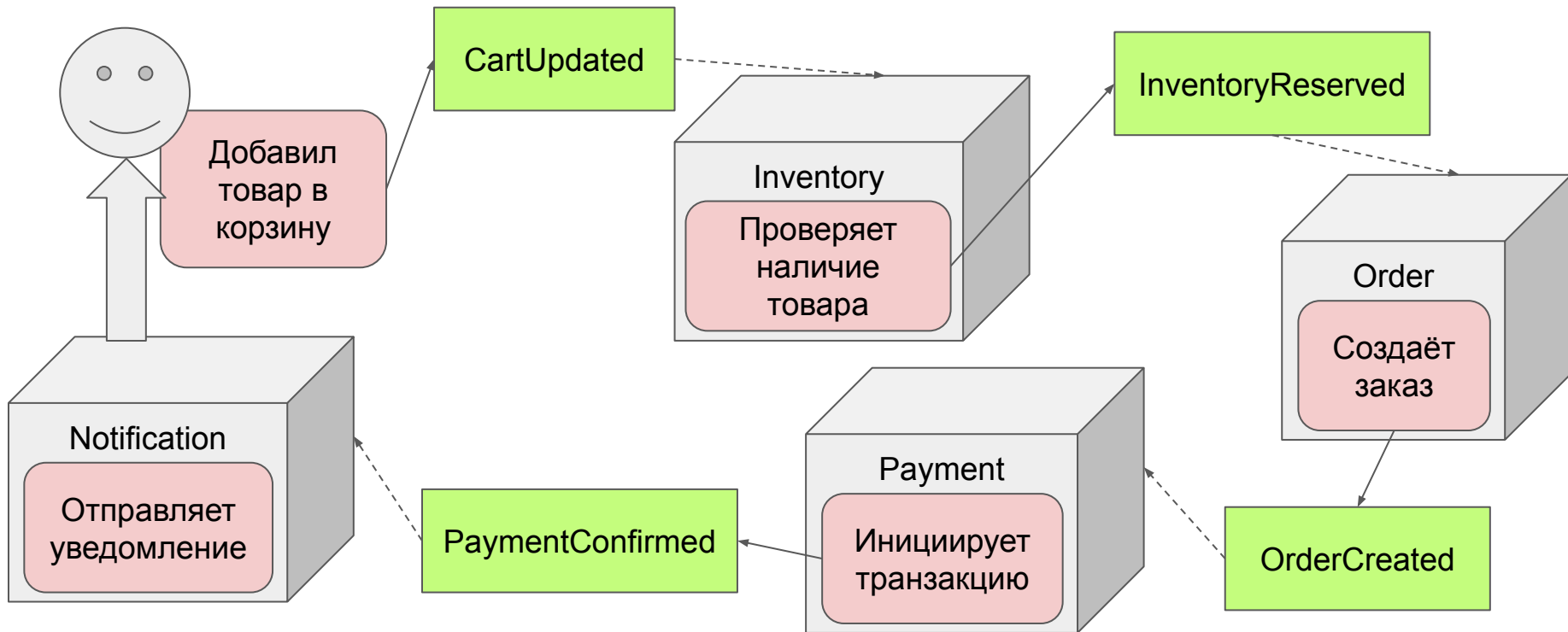
Архитектурные проекции

- Каждый BDD-сценарий можно спроецировать на архитектуру
- Вопросы для проекции:
 1. Какие компоненты участвуют?
 2. Какие данные нужны?
 3. Какие события происходят?
 4. Какие интеграции активируются?

Оформление проекций

1. **C4-диаграмма** отображает компоненты в контексте системы в целом
2. **Sequence-диаграмма** показывает поток и очерёдность вызовов, зависимостей между компонентами
3. **Event flow** показывает какие события публикуются, кто их слушает, где возможны задержки и/или конфликты

BDD и Event-Driven Architecture



Практическое значение

Архитектурная проекция даёт:

- Ясность - какие компоненты нужны и как взаимодействуют
- Прозрачность для бизнеса
- Раннее выявление рисков
- Поддержка тестируемой архитектуры

Вопрос для общего обсуждения

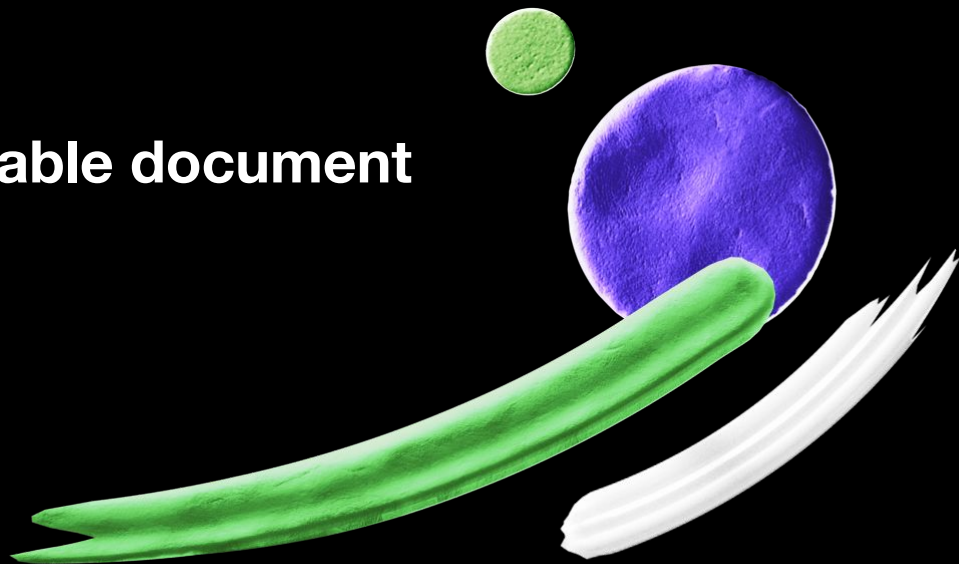
Какие инструменты отвечают на 4-й вопрос проекции?

2 минуты



IV. Спецификации и тестируемое проектирование

- 01 Спецификация как executable document
- 02 Инструменты BDD
- 03 Тестируемая архитектура



Исполнение спецификации

- BDD-сценарий может быть исполнен буквально
- Что это даёт на практике?
 - Проверка корректности поведения
 - Спецификация – часть репозитория
 - Обновление вместе с архитектурой
- Итого: документация – живой механизм **контроля качества** системы

Инструменты BDD

- Для исполнения документации нужна среда (фреймворк). Примеры сред:
 - Cucumber – Java, Javascript, Ruby
 - SpecFlow – .NET
 - Behave – Python

Тестируемая архитектура

- Интеграционные тесты
 - Проверяем взаимодействие между компонентами, а не работу самих компонентов
- Контрактные тесты
 - Проверяем корректность контракта
- Проверка бизнес-инвариантов

Почему всё это важно?

- “Живая” архитектура – согласованность требований и реальной системы
- Снижение рисков изменений – есть валидация бизнес-сценариев
- Коммуникация между командами – спецификация как общий язык
- Скорость и гибкость – поведение валидируется, поэтому легко добавлять новый функционал

Вопрос для общего обсуждения

Почему можно гарантировать, что ненамеренное изменение бизнес-логики однозначно будет определено тестами?

2 минуты



Вопрос для общего обсуждения

В чём основное отличие между DDD и BDD с точки зрения коммуникации между командами?

2 минуты



Сегодня мы узнали, как:

- 1 **Использовать BDD для описания требований через сценарии**
- 2 **Связывать сценарии с архитектурными решениями**
- 3 **Использовать спецификации для тестируемого проектирования**



Вопросы?